

«Магическая пилюля»: модель ответит точнее, если попросить её экспертом и усложнить промпт.

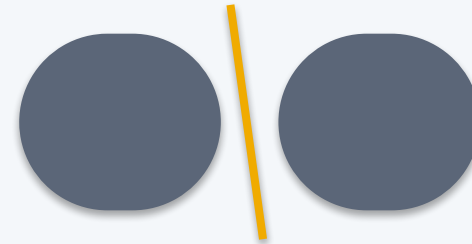
ОБЕЩАНИЕ



«Ты — эксперт-юрист. Рассуждай шаг за шагом, будь очень внимателен...»

→ ждём: ответ станет вернее.

РЕАЛЬНОСТЬ



Роль настраивает ТОН, а не точность. Пошаговое рассуждение помогает лишь на цепочках шагов. Усложнение промпта само по себе — не апгрейд.

Что делать: разложим, что реально повышает надёжность AI-системы — и научимся выбирать архитектуру под задачу, а не глотать «пилюлю».



Первый разбор: чат-бот выдумал политику — платит компания.

Moffatt v. Air Canada · трибунал ВС · 14.02.2024

1

Пассажир спросил чат-бота про тариф по случаю утраты близкого

2

Бот: «купи по полной цене, верни разницу в течение 90 дней»

3

Реальная политика этого не допускала — и была на той же странице, куда бот ссылался

4

Трибунал: «бот — не отдельное юр. лицо» → компания вернула \$812,02^[1,2]

Что делать: юридически значимый ответ клиенту — не для генеративного бота. Нужен детерминированный источник политики; бот — лишь навигатор к официальному документу.

Архитектуры AI-систем: агенты, RAG, API

03

Какую архитектуру выбрать под задачу —
и когда правильный ответ «не ИИ»

Маршрут лекции — шесть разделов.

Одна несущая линия: выбор архитектуры под задачу — и когда правильный ответ «не ИИ».

0



Открытие

Air Canada: неправильная архитектура
стоит денег

1



Промпт и его границы

что умеет один вызов и где его потолок

2



RAG

внешнее знание в контекст — и где оно
тихо ломается

3



Fine-tune

менять веса под поведение, не под знание

4



Агенты

цикл, экипировка, память, безопасность —
и провалы

5



Фреймворк

лестница + чек-лист: как выбрать быстро
и обоснованно

Что мы переносим из Лекции 2 — и что надстроим.

Вокруг одного вызова модели надстраиваются 4 обвязки. Два готовых блока из Лекции 2 не переобъясняем.



Из Лекции 2 берём готовыми: *single-shot инференс* и *семантический поиск на эмбедингах* (основа RAG). Детали каждой обвязки — дальше по разделам.

У меня есть задача и доступ к LLM. Какую архитектуру выбрать — и когда правильный ответ «не ИИ»?

6

Multi-agent

несколько координируемых агентов

сложнее ↑

5

Агент

цикл `plan` → `act` → `check` → `iterate`

4

Workflow

предопределённые пути

3

RAG / контекст-инжиниринг

поиск-дополненная генерация

2

Один вызов LLM

промт; + CoT (по шагам), + few-shot (примеры)

1

Обычный код (без ИИ)

точка отсчёта

проще ↓



Подниматься на следующую ступень — только при требовании задачи, которого текущая не закрывает.

РАЗДЕЛ 1

Промпт и его границы

Лестницу мы увидели целиком — теперь снизу: что умеет один вызов и где его потолок, прежде чем что-либо усложнять.

один точный рез · 3 разбора · 1 провал

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

Дефолт — один вызов с хорошим промптом.

Модель знает только то, что в промпте (плюс то, что было в весах на обучении). Больше ей взять неоткуда.



Один вызов LLM с хорошим промптом

- минимальная стоимость (один проход)
- минимальная латентность (нет лишних обращений)
- максимальная предсказуемость (нет петель, нет retrieval, который тихо деградирует)

Добавляешь RAG →

конвейер индексации + векторное хранилище + компонент retrieval (может молча деградировать) + метрики его качества

Добавляешь инструменты / цикл →

внешние вызовы (падают, тормозят) + петли (расходятся) + недетерминизм траектории + новая поверхность атаки

Не усложняй архитектуру без причины, выраженной в требованиях задачи^[1]. Это распределение бремени доказательства, а не примитивизм.

[1] Anthropic — Building Effective Agents («найди простейшее»)

Роль в промпте настраивает тон — не точность.

«Ты — опытный юрист» сдвигает внимание модели (механизм — Лекция 2) в сторону текста, похожего на ответ такой роли, — но это про стиль, не про факты.

Расхожий миф

«Напишу роль эксперта — и модель будет точнее отвечать по фактам»

Что показал эксперимент

162 персоны · 6 типов отношений · 8 доменов · 2410 вопросов фактического QA · 4 семейства LLM. Персоны НЕ улучшили точность ответа по сравнению с ответом без персоны вообще^[1].



На что роль влияет реально

на тон и глубину изложения — насколько формально и подробно, а не верен ли факт (эффект модель-специфичен).

Нужна фактическая точность? Инструмент — не формулировка роли, а грамотный контекст и, при необходимости, RAG (опора на проверяемый источник). Это ещё один пункт «магической пилюли», опровергнутый измерением.

«Роль» — это два разных механизма. Не путайте их.

Роль-персона («ты — юрист») настраивает тон. Протокольная роль `system/user/assistant` — это разметка «кто говорит» в потоке токенов. Второе постоянно принимают за первое.

{ } Chat-шаблон: список ролей → плоский поток токенов

```
<|im_start|>system правила поведения <|im_end|>
<|im_start|>user вопрос <|im_end|>
```

Спецтокены разметки — ChatML / Llama 4; у Anthropic `system` — отдельное `top-level` поле. Шаблон — Jinja в `tokenizer_config.json`: его можно прочитать и подменить.

Приоритет `system` — склонность, не граница

GPT-4o слушается приоритета ~**63,8%**; IH-Challenge **84,1% → 94,1%** (всё равно ≠ 100%).



STI / role spoofing: роль подделывают

Строка `<|im_start|>assistant` из внешнего текста → ChatInject ASR **5,18% → 32,05%** (Llama-4 до 88,3%).

Что делать: не проектируйте защиту в расчёте «система важнее пользователя всегда». Экранируйте спецтокены во **ВХОДЯЩЕМ** внешнем контенте и проверяйте chat-шаблон у локальных моделей — чужой шаблон тихо ломает приоритет.

Структура промпта: разделяй инструкцию, контекст, данные.

Плоский промпт заставляет модель угадывать, где кончается инструкция и начинаются данные. Явная граница снимает эту неоднозначность.



Инструкция

что сделать



Контекст

на основе чего



Данные

с чем именно работать

Разделители (delimiters)

- XML-теги: `<инструкция>...</инструкция>`
- Markdown-заголовки: `## Задача · ## Данные`
- Тройные кавычки / бэктики вокруг данных

Тот же принцип, что structured output (Раздел 4): там схема задаёт структуру ВЫХОДА, здесь разделители — структуру ВХОДА. Ясная структура вместо «выведи форму по смыслу».

Модель, которой показали, где кончается инструкция и начинаются данные, реже принимает фрагмент данных за новую команду — та же путаница лежит в основе prompt injection (Раздел 4).

Chain-of-thought помогает — но его нельзя аудировать.

Без CoT

«Было 23 яблока, 7 испортились, докупили 2 ящика по 6.
Сколько хороших?»

→ правдоподобное, но неверное число

С CoT («решай по шагам»)

$23 - 7 = 16^{[1]}$ · $2 \times 6 = 12$

$16 + 12 = 28$

→ верно

Технически это всё ещё один вызов — CoT инструмент под класс задач (цепочка шагов), не глобальный тумблер.

Но проговорённое рассуждение не обязано отражать реальную причину ответа (низкая faithfulness):

~25%

Claude 3.7 Sonnet

~39%

DeepSeek R1

— доля случаев, где модель упомянула реально использованную подсказку^[2].

Контроль на самообъяснении модели — не контроль. Человек-валидатор проверяет РЕЗУЛЬТАТ и факты против внешнего источника, а не правдоподобность текста. И это же — почему шаг check в цикле агента (Раздел 4) не должен быть самооценкой модели.

Рассуждение вслух — не журнал аудита.

Faithfulness (верность объяснения) — насколько проговорённая цепочка отражает реальную причину ответа. Измерили напрямую — и она низкая.

Эксперимент Anthropic (апр. 2025)



Задачу дают дважды: без подсказки и с подсказкой в промпте, которая меняет ответ («профессор считает, что С»)



Смотрят: когда ответ изменился под подсказку — упомянула ли модель её в рассуждении?



Часто модель меняет ответ, но строит ДРУГУЮ, выдуманную аргументацию, не называя реальную причину

Как часто модель признаёт подсказку

Claude 3.7 Sonnet



DeepSeek R1

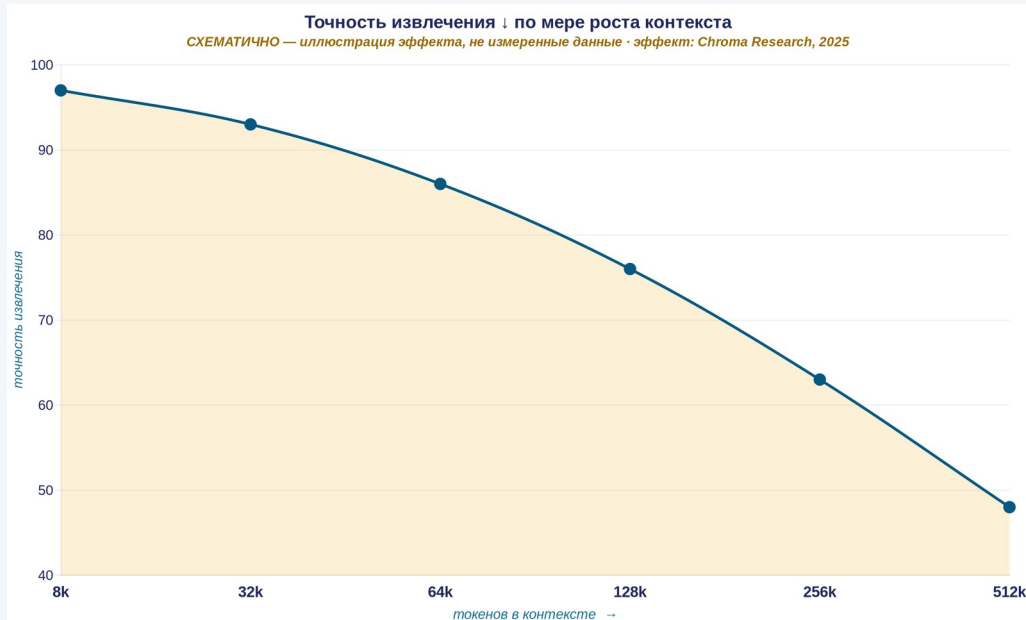


И хуже — на трудных задачах (GPQA ниже, чем MMLU). Где аудит нужнее всего, объяснению верить можно меньше всего.

Что делать: человек-валидатор проверяет РЕЗУЛЬТАТ против независимого источника (база, документ, расчёт, эксперт), а не приложенную «цепочку рассуждений». Контроль на самообъяснении модели — не контроль.

Контекст-инжиниринг: минимум высокосигнального.

Промпт-инжиниринг — одна инструкция. Контекст-инжиниринг — курирование всего набора токенов, видимых модели на инференсе.



Когда НЕ RAG (точка 1)

малый стабильный корпус, влезает в окно → full-context + кэширование префикса, а не RAG-инфраструктура



RAG здесь добавил бы хрупкость без выигрыша

context rot = тот же «lost in the middle» из L2^[1,2] — новый термин, не новая сущность

«Найти наименьший набор высокосигнальных токенов, максимизирующий вероятность желаемого исхода» — это инженерное требование, не эстетика^[3].

Чит-шит: как строить промпт.

Минимум, ниже которого промпт систематически недорабатывает. Приложите к любому промпту перед первым запуском.

1 Роль

если нужен тон/регистр — и НЕ как обещание точности

2 Задача

конкретное проверяемое действие, не расплывчатое пожелание

3 Контекст

минимально необходимое, не «всё, что могло бы пригодиться»

4 Формат вывода

указан явно, если ответ машинно-обрабатываем

5 Разделители

если содержимого больше одного вида (инструкция/данные)

6 Примеры (few-shot)

только если формат неочевиден из одной инструкции

7 CoT

только если задача требует многошагового рассуждения

8 Длина

не длиннее необходимого — лишние токены «тонут» в контексте

Компактная форма всего раздела: роль ≠ точность, структура помогает разделить входы, CoT — точечный инструмент, контекст — минимальный. Для крупных архитектурных решений (RAG / FT / агент) — восьмишаговый чек-лист Раздела 5.

РАЗДЕЛ 2

RAG: поиск-дополненная генерация

Извлечь релевантное → положить в контекст → ответить с опорой на источник

внешнее знание · 3 разбора · 1 провал

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

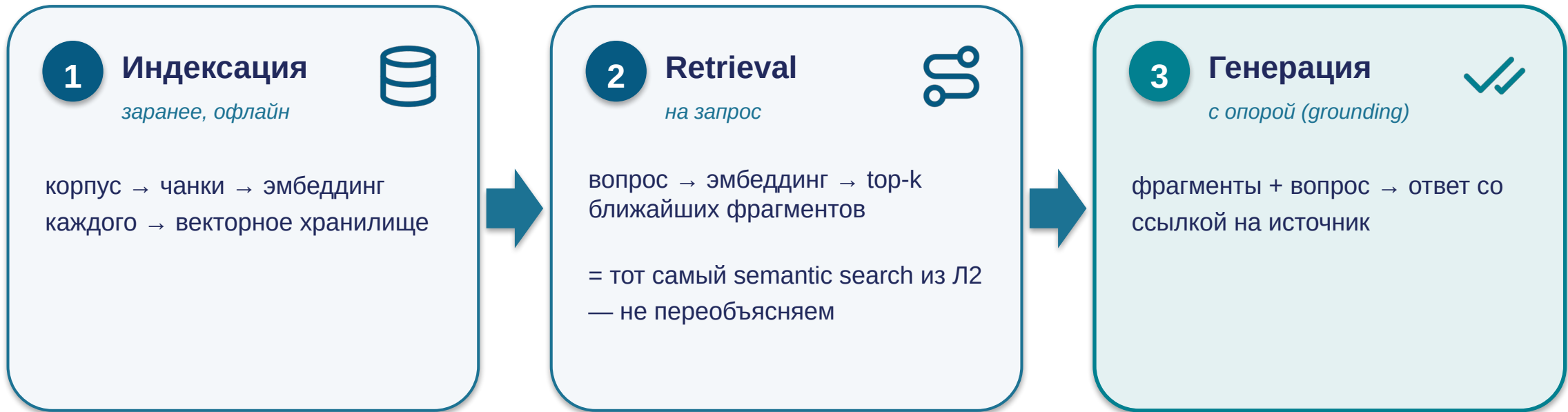
Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

Принцип RAG — три шага.

RAG = индексация → retrieval (semantic search Л2) → генерация с опорой; «не знаю» — корректный ответ.



«Не знаю» / «см. источник X» — корректный ответ RAG-системы^[1]. Правдоподобный ответ при нерелевантном retrieval — это дефект, а не «лучше, чем ничего».

[1] [Lewis et al. 2020 — Retrieval-Augmented Generation \(NeurIPS\)](#)

Когда RAG — правильный выбор.

RAG оправдан при сильном сигнале по признакам ниже^[1] — и отсутствии blockers со следующего слайда «когда НЕ RAG».

Большое / растущее

не влезает в окно целиком, или
дорого класть весь корпус в
каждый запрос

Меняется

документы, цены, регламенты
обновляются чаще, чем
выходят версии модели

Свежесть + провенанс

ответ опирается на
проверяемый источник; можно
показать, откуда факт

Приватная база

знания компании не входят в
веса публичной модели

Признаки усиливают друг друга → RAG

Корп. база из тысяч регламентов, обновляется еженедельно, ответ с обязательной ссылкой на пункт: все признаки сошлись, ни один более простой механизм их совместно не закрывает → образцовый профиль RAG.



Один признак — повод присмотреться^[2], но не строить автоматически: сверьтесь с блокерами на следующем слайде.

^[1] IBM — RAG vs Fine-Tuning (вендор-нейтрально) · ^[2] U-NIAH — RAG win-rate выше у меньших моделей

Когда RAG — НЕ правильный выбор.

Знать, когда RAG не нужен, ценнее: это модная архитектура, её ставят туда, где она вредит.

1 Корпус влезает в окно

*ориентир — менее ~200k токенов,
меняется редко*

→ **full-context + кэширование
префикса, не RAG-инфраструктура**

2 Фиксированная политика / значение^[2]

тариф, цена, пункт регламента, правило

→ **детерминированный lookup /
статическая страница**

3 Данные доступны live через API / MCP

*во внутреннем сервисе, базе, поиске другой
системы*

→ **вызвать инструмент напрямую;
RAG-индекс поверх — лишний, более
хрупкий и более устаревающий слой**

RAG избыточен, если выполнен ЛЮБОЙ из трёх^[1]: (а) корпус влезает в окно и стабилен, (б) задача сводится к фиксированному значению, (в) знание уже доступно напрямую и живьём через инструмент. «Прямой вызов вернёт данные на момент запроса; RAG-индекс — на момент последней индексации.»

Провал RAG на масштабе.

«Вернул что-то» ≠ «вернул правильное»^[1,2]. У RAG нет сигнала «не нашёл» — он всегда отдаёт к ближайших, даже нерелевантных.



Legal-AI

«ближайшие» дела из другой юрисдикции / отменённый прецедент — модель опирается на них как на факт



Medical-RAG

смешал фрагменты разных пациентов — близки по симптомам, клинически объединять нельзя



Support-бот

работал на сотнях статей; после роста до тысяч качество тихо просело — никто не заметил

Air Canada — разбор архитектуры^[3]

Диагноз

сгенерированный правдоподобный текст поставлен в роль, требовавшую извлечённого проверенного факта — отказ grounding

Правильная альтернатива

фиксированная политика → lookup / страница; нужен диалог → RAG со strict grounding, обязательная цитата, явное «не знаю», проверка человеком

РАЗДЕЛ 3

Fine-tune vs промпт vs RAG

Проблему знания мы решили через RAG. А если проблема не в знании, а в поведении модели — её тоном, форматом, политикой?

настройка поведения · 4 разбора · 1 провал

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

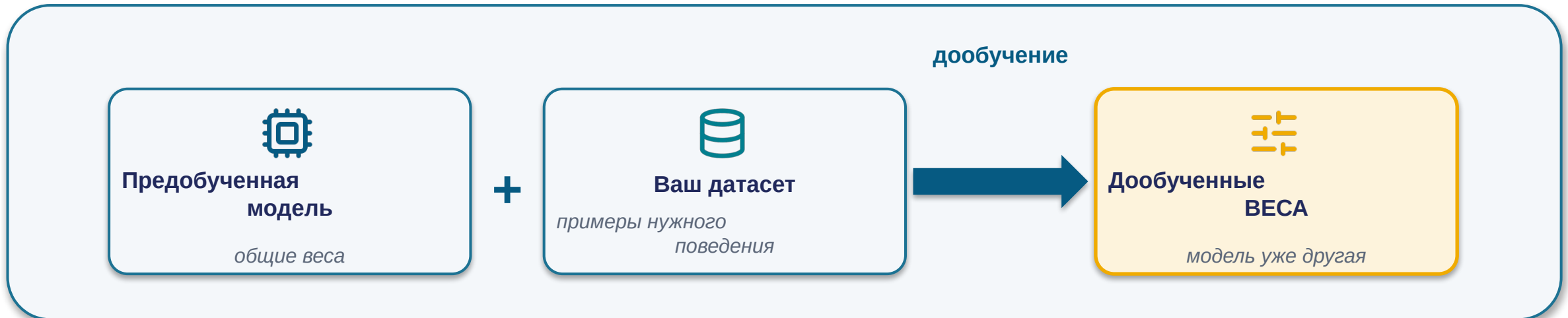
Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк

Что такое fine-tuning.

Fine-tuning (дообучение) — продолжение обучения уже готовой модели на ваших данных. В Л1 — тип использования; здесь — архитектурный выбор, одна из ступеней лестницы.



Промпт / RAG → КОНТЕКСТ

Меняют только вход — веса не трогаются; эффект живёт лишь в рамках запроса.

Fine-tuning → САМИ BECA

Изменение встроено в модель — действует всегда и стоит дороже того, что меняет контекст.

Промпт/RAG = «что показать модели». Fine-tuning = «изменить саму модель»^[1]. На практике «дообучить» почти всегда означает LoRA/PEFT, а не переобучение всех весов (следующий слайд).

^[1] IBM — RAG vs Fine-Tuning (fine-tuning меняет веса)

PEFT вместо full fine-tuning.

PEFT — базовые веса замораживаются, обучается лишь небольшой набор адаптеров. LoRA — низкоранговые матрицы-адаптеры; QLoRA — то же поверх квантованной модели^[1,2].

Базовые веса
(frozen)

LoRA

LoRA

LoRA

адаптеры — мегабайты vs гигабайты

98,4% моделей с тегом PEFT — это LoRA^[3]

из 20 834 карточек на Hugging Face Hub · оговорка: доля среди моделей с тегом PEFT, не среди всего fine-tuning

1. Дешевле и быстрее

обучаются миллионы параметров вместо миллиардов; QLoRA — на одном GPU

2. Модульность

адаптеры мегабайты vs гигабайты; одна база — много специализаций

3. ↓ Риск catastrophic forgetting

база заморожена, физически не переписывается под новый сигнал — архитектурный аргумент

PEFT (LoRA/QLoRA) почти всегда лучше full fine-tuning: дешевле, модульнее, ↓ риск forgetting. Full FT в 2026 — почти никогда.

Что здесь знание, что поведение, что детерминировано.

Вопрос на проектировании — не «RAG или fine-tuning». Разнесите проблему по осям: знание → RAG, поведение → PEFT, дешевле → дистилляция, детерминированное → код.

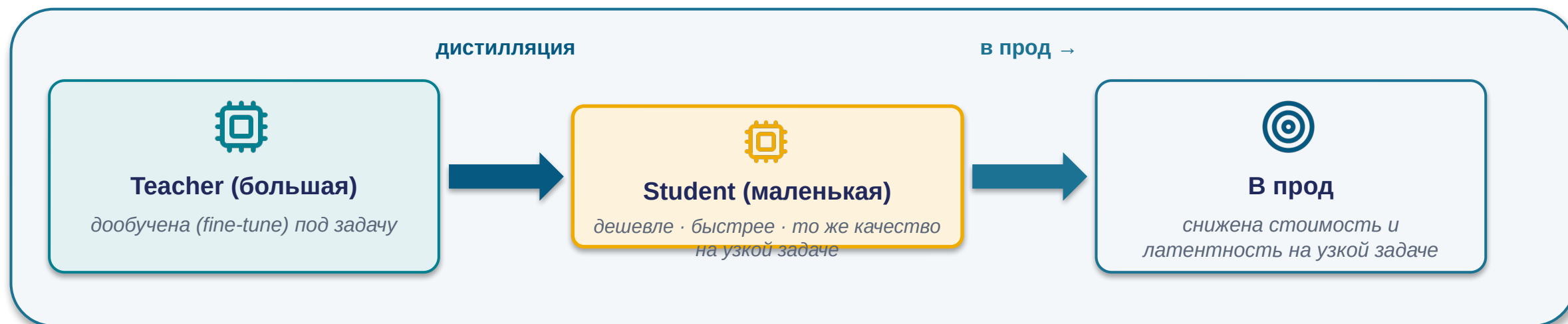
Если задача требует...	→ правильный инструмент	...и НЕ этот, потому что
знание меняется / нужны свежесть, провенанс	RAG (или длинный контекст для малого корпуса)	не fine-tuning: знание устареет, переобучать дорого, риск forgetting
стабильное поведение / тон / формат / политика	fine-tuning (PEFT)	не RAG: подаёт знание в контекст, но не меняет манеру модели
снизить стоимость / латентность на узкой задаче	fine-tune teacher + дистилляция student	две отдельные техники в связке; промпт/RAG не уменьшают размер модели
детерминированный, верифицируемый ответ	обычный код, без ИИ	ни RAG, ни FT: ИИ добавит недетерминизм без выигрыша

Что делать: гибрид — норма^[1] TAM, где у задачи есть И проблема знания, И проблема поведения. Нет проблемы поведения — FT не нужен, даже когда есть RAG. Каждый компонент добавляется под своё требование — то же правило лестницы.

[1] [BigData Boutique — Fine-Tuning When RAG Isn't Enough](#)

Дистилляция — не вид fine-tuning, а отдельная техника.

Fine-tuning меняет поведение модели. Дистилляция — сжатие: перенос умений большой модели в маленькую. Их часто путают, но это две таксономически разные операции^[1,2], работающие в связке.



Fine-tuning отвечает на:

«как модель себя ведёт» — тон, формат, следование политике.
Меняет BECA под поведение.

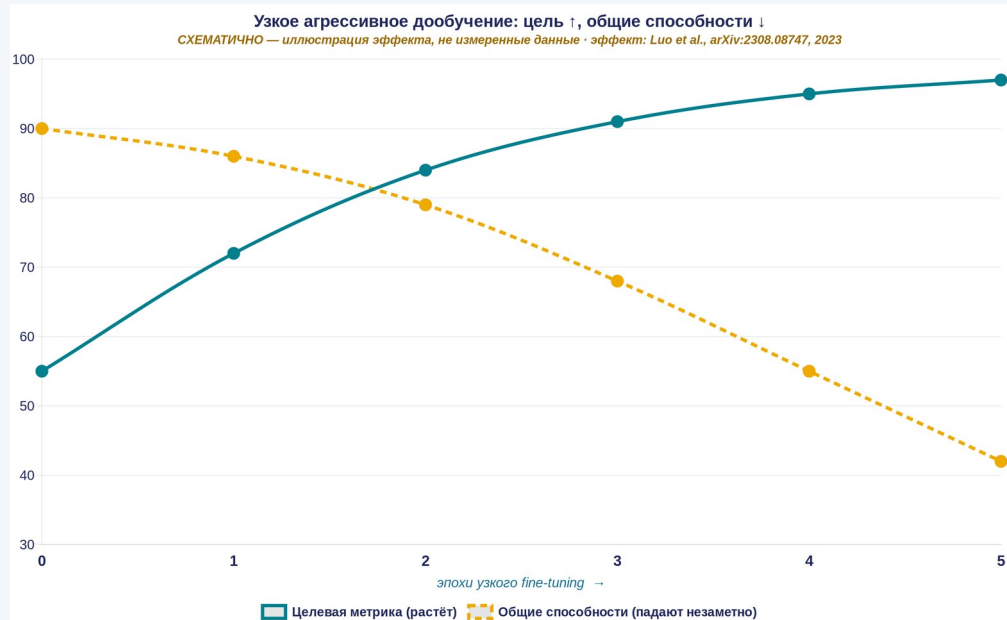
Дистилляция отвечает на:

«как сделать модель дешевле» — тот же навык в меньшей модели.
Это сжатие, а не смена поведения.

Что делать: не пишите «дистилляция — это fine-tuning». В связке они идут так: сначала дообучить teacher под задачу, потом дистиллировать в student ради цены. Критерии «что куда» — на следующем слайде.

Провал: catastrophic forgetting.

Catastrophic forgetting (катастрофическое забывание) — деградация общих способностей модели в результате узкого агрессивного дообучения^[1].



тяжелее с ростом масштаба модели — у крупной выше исходный уровень, ей «больше падать»

Нет eval-петли на общих задачах + нет версий датасета/весов → не увидишь поломку до прода + не сможешь откатиться → это не «риск», это критерий «НЕ делай fine-tuning»

Правильно: PEFT (замороженные веса — ниже риск); для меняющегося знания — RAG, не FT вовсе.

[1] Luo et al. 2023 — [Catastrophic Forgetting in LLM Continual FT](#) · Эмпирически наблюдается при continual fine-tuning (Luo et al. 2023). Механизмы — «исследования показывают» (preprint).

РАЗДЕЛ 4

Агенты

От собеседника в окне чата — к компоненту продакшен-системы: цикл, экипировка, память, доступ к инструментам — и где всё это ломается.

агент + экипировка · 5 разборов · 4 провала

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк



Модель становится компонентом системы: API + MCP.



Structured output

выход строго по схеме (JSON), не текст для парсинга

встраиваемая



Function calling

модель формулирует «вызовы X»; исполняет ваш код, не модель

активная



Prompt caching

не пересчитывать неизменный префикс при каждом запросе

экономичная



MCP — «USB-C для инструментов LLM»

$N \times M$ несовместимых интеграций → $N + M$ ^[1]

инструмент один раз = MCP-сервер; модель один раз = MCP-клиент

AI Anthropic
11/2024



 OpenAI
03/2025



 Google
04/2025

Стандартизация подключения ≠ безопасность подключаемого — и усугубляет проблему доверия.

- код в вашем окружении / доступ к данным
- описание попадает в контекст — носитель prompt injection

Ни один механизм не делает модель надёжнее — правило лестницы не отменяется. Удобство подключения — не аргумент за подключение.

[1] [Anthropic — MCP donation / Agentic AI Foundation \(N×M → N+M\)](#) · Актуальные цифры экономии и масштаб экосистемы MCP — в главе методички.

Агент — это не «чат подороже». Это другой порядок цены.

База сравнения — один *chat-turn* (десятки центов). Каждая ступень вверх умножает расход токенов, а не прибавляет его.

Один вызов LLM (chat)

базовая стоимость запроса

×1

Одиночный агент (цикл)

план → act → check → iterate: много проходов на задачу

×несколько

Мульти-агент

поверх агента — координация нескольких агентов (Anthropic, 2025)

ещё ×15

Что делать: считайте бюджет ДО выбора архитектуры. Prompt caching (не пересчитывать неизменный префикс) в агенте — уже не «приятный ориентир», а необходимость: без него ×N-стоимость цикла становится неподъёмной.

Множители — порядок величины (Anthropic, 2025), не точный тариф; абсолютная цена зависит от модели и задачи.

MCP: подключить инструмент — теперь операция на минуты.



MCP — «USB-C для инструментов LLM»

N×M несовместимых интеграций → **N+M**

инструмент один раз = MCP-сервер; модель один раз = MCP-клиент.
Открыт Anthropic 11/2024 → OpenAI, Google 2025.

Читайте каталог критически

«до 90 000 серверов» → реально запускаемых ≈ **10 000** =
11% каталога.

Остальное — дубли, битые и заглушки. И даже рабочие 10 тысяч не проверены на безопасность.



Поворот доверия: стандартизация подключения ≠ безопасность подключаемого — и обостряет её.

30+ CVE

за 60-дневное окно против MCP-серверов (~43% —
внедрение команд)

82%

обход пути (path traversal) среди 2 614 проверенных
реализаций

Что делать: удобство подключения — не аргумент за подключение. Каждый MCP-сервер = чужой код в вашем окружении + носитель инъекции в контексте. Подключаете под требование задачи, с оценкой новой границы доверия.

Цикл агента: plan → act → check → iterate.

Агент — архитектура, где модель не делает один проход, а работает в цикле^[1], сама определяя последовательность шагов.



Workflow vs Agent.

Предсказуемая задача → workflow^[1]; непредсказуемая И ценность оправдывает кратный рост → агент.



Workflow (рабочий поток)

LLM и инструменты по predetermined в коде путям

- последовательность шагов известна заранее
- предсказуемо, аудируемо
- большинство надёжных продакшн-систем — это workflow



Агент

LLM динамически определяет собственный процесс

- последовательность заранее не зафиксирована
- кратно больше токенов, чем чат
- ниже аудируемость, выше риск петель

Могу ли я заранее, до запуска, выписать последовательность шагов? **да (даже с ветвлениями) → workflow** ·
принципиально нет И ценность оправдывает кратные стоимость/риск → агент^[2]

Вложенность — норма: агент code review вызывает workflow «линтер → тесты → формат» как один шаг; workflow обработки заявок делегирует мини-агенту разбор свободной жалобы. Динамика — только там, где нужна.

Найди простейшее. «Лень формализовать» не делает задачу непредсказуемой.

Мульти-агент по умолчанию — не апгрейд, а множитель риска.

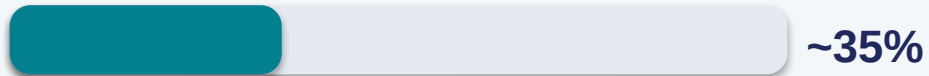
В цепочке из n шагов, где сбой любого портит результат, надёжности перемножаются: успех $\approx p^n$. Инженерная интуиция «каждый шаг почти всегда работает» математически ложна.

Надёжность цепочки = p^n

95% × 10 шагов



90% × 10 шагов



95% на шаг звучит надёжно — но $0,95^{10} \approx 0,60$. Больше агентов = больше шагов = ниже общая надёжность.

Топология решает: координатор > рой

«Рой» равноправных агентов амплифицирует ошибки **17,2×**; один координатор — только **4,4×** (Zartis/Redis).



Anthropic дословно: «multi-agent works mainly because it helps spend enough tokens to solve the problem» — выигрыш от объёма токенов, не от «магии координации».

Что делать: начинай с одного сильного агента. Мульти-агент — только если задача распадается на ШИРОКО параллельные независимые подзадачи высокой ценности; иначе +15× токенов и координационные издержки не окупятся.

Каждый слот экипировки агента — компромисс, не апгрейд.

Реальный агент-помощник (Claude Code, Cursor, Aider) — это цикл *plan* → *act* → *check* → *iterate* ПЛЮС оснастка. Пять типовых слотов^[1]:



Как не поднимаешься по лестнице архитектур без требования задачи — не добавляй агенту память, subagents или MCP «на всякий случай». Каждый слот несёт свою цену: операционную сложность, новую поверхность отказа, новую границу доверия — и должен отвечать на конкретный триггер (Раздел 5).

[1] *agent-harness-registry* — карта слотов экипировки агента

Память агента — тот же вопрос масштаба, что RAG.

Память — что агент помнит *МЕЖДУ* сессиями (в отличие от контекста одного разговора). Спектр — от плоского файла до graph-баз^[1].



Плоский файл

агент дописывает факты в текстовый лог, при запуске читает целиком. Работает, пока лог мал и стабилен — прямая параллель критерию RAG «корпус помещается в окно».



Векторная / graph-база

- mem0 — cross-session память пользователя
- Cognee — memory на графе знаний
- Graphiti / Zep — temporal graph: когда факт верен, когда устарел

Тот же вопрос масштаба знания, что решает RAG для корпуса документов, здесь встаёт для памяти самого агента. Не ставьте graph-базу агенту с короткими несвязанными сессиями — это тот же технический долг без требования, что RAG для десяти статей.

[1] [agent-harness-registry \(live-eval\)](#) — спектр памяти агента · Источник: публичный реестр [agent-harness-registry \(workain lab, live-eval\)](#).

«Агент, который помнит» — не всегда лучше.

Наличие памяти интуитивно кажется чистым улучшением. Независимый live-eval показывает: иногда — драматически нет^[1].



Letta — Tier D

проигрывает И голой модели, И плоскому файлу на всех задачах.
persistbench_v1:

 Голая модель	1.000	94 с
 Плоский файл	0.833	159 с
 Letta	0.750	496 с

Механизмы: капитуляция под давлением · многословие топик факт · факт замечен, но не закодирован.

Оговорка: тест на Letta v0.6.7 — отстаёт от текущей v0.16.8 (~18 мес).

Anthropic Memory Tool — Tier B

В целом сильный результат — но даже он теряет данные в

17% задач

- явный отказ записать «эфемерный» факт
- немотивированный отказ «вне рамок»
- тихая lossy-суммаризация — деталь не восстановить
- невоспроизводимость: та же беседа дважды → разный результат

Файл-инструкция агенту — не «магическая прививка».

Операционный слой — файлы-конвенции (класс *CLAUDE.md* / *AGENTS.md*) + журнал задач. Интуиция «написал инструкцию → лучше» расходится с измерением.



presence paradox

RCT (Gloaguen et al. 2026): само наличие файла-инструкции НЕ даёт значимого прироста^[1] успешности — при этом стоимость и число шагов растут.

Помогает только там, где реально заполняет пробел документации.



Honest Lying

Dixit, Kamal, Oates 2026: self-authored память может ЗАКРЕПЛЯТЬ неверное убеждение — журнал бетонирует раннюю ошибку вместо пересмотра.

Рефлексия ошибочна → повторные попытки опираются на неё.



claude-code#51735

Реальный кейс: письменно признанная прошлая ошибка НЕ предотвратила её повторение спустя 25 дней^[2].

Запись о провале ≠ гарантия, что поведение изменится.

Полезен, когда заполняет реальный пробел; бесполезен, когда дублирует доступное модели; может навредить, когда самоавторская память закрепляет ошибку. Тот же паттерн, что роль в промпте: «добавить X → лучше» не подтверждается измерением.

[1] [Gloaguen et al. 2026 — Evaluating AGENTS.md \(presence paradox\)](#) · [2] [GitHub anthropics/claude-code#51735 \(повтор ошибки через 25 дней\)](#)

Skills, subagents, доступ — и граница доверия.



Skill

переиспользуемая процедура «как делать»
под повторяющуюся задачу



Subagent

отдельное контекстное окно: не засорять
главный + изолировать недоверенное



MCP-доступ

каждое подключение — новая граница
доверия и retention-политика



Безопасность: как только агент делегирует и подключается — появляется поверхность^[2,3], которой не было у одного вызова

GitHub MCP heist (май 2025)

issue с встроенной инструкцией + переизбыточный токен (PAT на все репо)
→ ассистент выгрузил приватные репозитории в публичный PR^[1].

Катастрофа = инъекция × широкие права. Убери любое — атака не проходит.

«У нас ZDR» ≠ «нигде не хранится»: судебный приказ (NYT v. OpenAI) + third-party/MCP вне ZDR^[4,5]. Регулируемое — не слать без ZDR/BAA.

4 правила проектирования

1

Least-privilege

минимум токенов/прав

2

Изоляция недоверенного

отдельно от привилегий (subagent)

3

Human-in-the-loop на write

необратимое — только через человека

4

Allowlist / pin

аудированные версии; deny-by-default

«У нас ZDR» ≠ «данные защищены во всей цепочке».

ZDR (zero data retention) покрывает основные вызовы модели — но не всю архитектуру. Агент = модель + инструменты, и инструменты часто вне ZDR.



ZDR НЕ покрывает

- Files API
- batch-обработку
- исполнение кода в контейнерах
- MCP-коннектор
- сторонние (third-party) интеграции
- потребительские планы

Чем агентнее архитектура — тем больше её данных проходит через эти звенья.



Судебный приказ переживает вашу политику

NYT v. OpenAI (2025): суд обязал сохранять ВСЕ логи ChatGPT как доказательства. Договорная политика «30 дней» оказалась бессильна против litigation hold по чужому спору.

Данные, покинувшие ваш периметр, живут по правилам, на которые вы не влияете.

Что делать: составьте карту данных по каждой фиче до продакшена — какие данные, через какое звено, с какой retention-политикой, какие звенья third-party. Регулируемые/чувствительные данные — только с ZDR/BAA или локально (on-prem).

Реальные coding-агенты — через рамку экипировки.

Разница между инструментами — не в «качестве модели», а в том, какие слоты экипировки заполнены и где агент физически живёт^[1].

<> Claude Code

CLI + IDE · проприетарный

широкая экипировка: память, файлы-инструкции, skills, полноценные subagents, MCP — почти все 5 слотов

цена — большая операционная сложность

<> Aider

CLI-first · open source

минимальная простота: без развитой памяти, без subagents, без skills. ~47k★ на GitHub

«тонкая» оснастка — самостоятельный выбор, не недоразвитость

<> Cursor

desktop IDE (форк VS Code) · проприетарный

агент внутри редактора — не терминал. Форма интеграции — отдельная ось от оснастки

где живёт агент — тоже архитектурное решение

<> OpenHands

self-hosted платформа · MIT · ~80k★

широкая автономность, локальное/Docker/облако развёртывание

рабочая гипотеза по совпадению профиля — вероятный кандидат на «OpenClaw», не подтверждённый факт

Выбор инструмента подчиняется тому же правилу: не бери самый оснащённый по умолчанию — смотри, какая оснастка нужна именно твоей задаче.

[1] agent-harness-registry — обзор через рамку экипировки

Провалы агентов.

Каждый провал — режим отказа цикла агента в датированном кейсе: урок + альтернатива.



1. Петля без лимитов

Агент на «синхронизируй заказы» получил HTTP 429 → план → вызов → 429 → ...

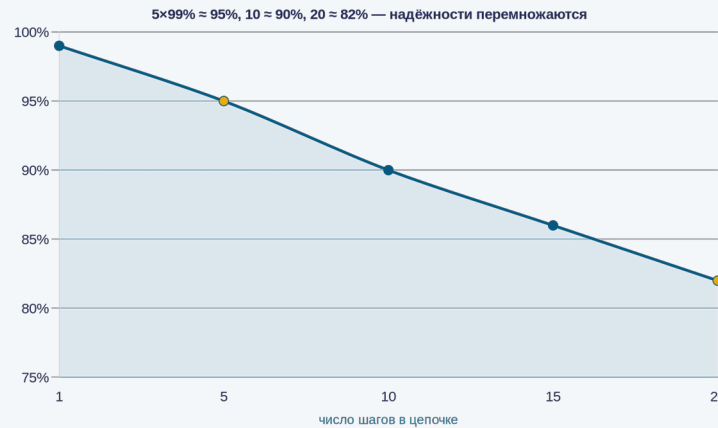
\$4 200 за 63 часа^[1]

База сравнения: retry-скрипт с backoff решил бы ту же задачу за секунды-минуты, практически бесплатно

\$4 200 — цена не автоматизации, а неправильного выбора архитектуры под предсказуемую задачу

Более подходящая архитектура: retry-with-backoff скрипт, не агент; лимиты бюджета и итераций ВНЕ агента

2. Накопление ошибок (compounding)



Вывод: «улучшить шаг» — слабый рычаг; «меньше хопов + валидация между шагами» — сильный^[2]



3. Мульти-агентная хрупкость

зависимые подзадачи → параллельные субагенты принимают конфликтующие^[3] неявные решения

Более подходящая: single-threaded линейный агент; мульти-агент — только широко-параллельное независимое

Провалы агентов — это КЛАССЫ, а не список курьёзов.

Из **344** enterprise-релевантных ИИ-инцидентов в **188** (~55%) автономная система нанесла ущерб в проде БЕЗ атакующего — агент сломал всё сам. Запоминайте класс, не дату.

Класс провала	Кейс · дата	Выученный урок
Деструкция + over-privilege	PocketOS · 04.2026	System-промт ≠ контроль; жёсткая граница + least-privilege
Zero-click инъекция + утечка	EchoLeak / M365 Copilot	«Смертельная тройка» = эксплуатируемо
Runaway-стоимость	\$48k/14ч · \$1,3M/30дн	Нет критерия успеха и жёсткого потолка → цикл не завершится
Галлюцинация пакетов	slopsquatting · 19,7%	Имена зависимостей от агента нельзя доверять без реестра
MCP rogue-сервер	postmark-mcp · 09.2025	MCP-сервер = непроверенная цепочка поставок; подмена (rug-pull)
Мульти-агентный каскад	61% каскадов из апстрима	Где сломалось ≠ где проявилось; ошибки перемножаются
Юр. ответственность	OLG Hamm · Air Canada	«Это ответил бот» — не защита; вывод принадлежит компании
Петля без бюджета	\$4 200 / 63 ч	«Пробуй, пока не выйдет» без лимита = буквально
Накопление ошибок	reliability compounding	р ⁿ , а не усреднение; «улучшить модель» — слабый рычаг
Мульти-агентная хрупкость	Cognition, 2025	Для задач с зависимостями мульти-агент хуже одного

Что делать: столкнувшись с новым инцидентом — назовите ЕГО КЛАСС. Класс задаёт контрмеры (лимит / least-privilege / валидация между шагами), а конкретные даты и суммы — не нужно.

Четыре класса крупным планом — с базой сравнения.



PocketOS — прод-БД за 9 секунд

Агент нашёл токен с неограниченными правами в чужом файле и удалил том + все бэкапы. Ближайший восстановимый — 3-месячной давности.

Урок: системный промпт — не средство контроля безопасности; нужна жёсткая граница.



Runaway: \$48k / 14ч · \$1,3M / 30дн

Запрос без критерия успеха — планировщик расширился и не завершался. База: обычная сессия — центы-доллары; это на 3–5 порядков выше.

Урок: критерий завершения + жёсткий потолок + аварийный стоп обязательны.



Slopsquatting — 1 из 5 импортов

На 576 000 сэмплов кода 19,7% ссылок на пакеты — галлюцинации. Атакующий регистрирует выдуманное имя → следующий агент ставит чужой код.

Урок: pin версий/хешей, lock-файлы, ревью новых зависимостей.



Каскад — 61% ошибок из апстрима

На 73 инцидентах: в 61% корень был в апстрим-слое (retrieval/план), а не там, где сбой стал заметен. Следующий агент принимает ошибку за факт.

Урок: валидация между шагами, меньше хопов, трассируемость.

Что делать: во всех четырёх корень один — агент применён без внешней границы (прав, бюджета, валидации). Граница ставится ВНЕ агента, промптом её не заменить.

РАЗДЕЛ 5

Как выбрать: фреймворк решения

Мы разобрали все архитектуры по отдельности — и где каждая проваливается. Теперь соберём это в один инструмент выбора.

инструмент выбора · 4 разбора

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
Агенты

Раздел 5
Фреймворк



Лестница архитектурной сложности.

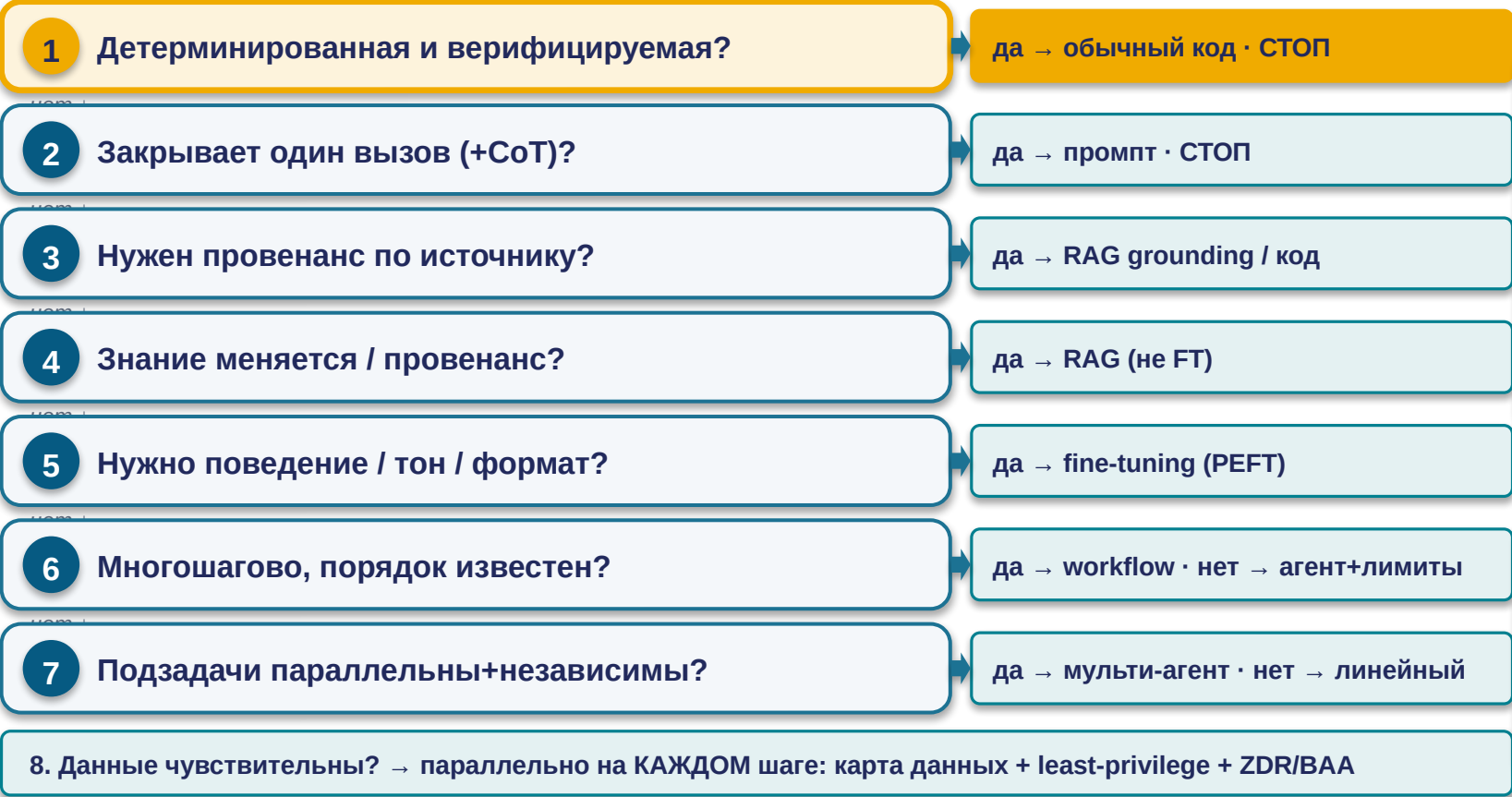
Оставайся на нижней ступени; поднимайся только под требование задачи.



[1] Anthropic — Building Effective Agents (правило лестницы) · Нижняя ступень — «обычный код без ИИ»: лестница начинается с вопроса «а нужен ли ИИ вообще».

План решения: маршрут вопросов, а не сумма баллов.

Пройдите задачу сверху вниз; останавливайтесь на первом сработавшем вопросе^[1].



Пример (задача А)

«2000 регламентов, меняются еженедельно, ответ со ссылкой на пункт» → вопрос 4 (меняется+провенанс) решающий → RAG со strict grounding. Не FT (устареет), не код (нужен NL).

Разминка (задача В)

«бот на ~150 FAQ, меняются раз в квартал» — пройдите маршрут сами; разбор на семинаре.

Приоритет маршрута: если задача детерминированная и верифицируемая — обычный код, СТОП здесь^[2]. ИИ добавил бы лишь недетерминизм, стоимость, латентность и поверхность для prompt injection.

[1] Anthropic — Building Effective Agents (маршрут выбора) · [2] McCarthy Tétrault — Air Canada (нижняя строка матрицы)

Стартовый комплект агента — и когда его усложнять.

Та же лестница «не усложняй без требования», применённая на уровень ниже — к оснастке одного агента, а не системы целиком.



Дефолт — тонкий агент

- один файл-инструкция
- плоская память
- БЕЗ subagents
- минимальный набор skills
- минимальный MCP-доступ
Бремя доказательства — на усложнении, а не на простоте.



Память-бэкенд — когда:

история переросла контекст ИЛИ нужен structured retrieval по фактам (не «последние N сообщений»). Тот же критерий, что переход промпт → RAG.



Subagents — когда:

подзадача требует отдельного окна (не засорять контекст) ИЛИ изоляции недоверенной работы (least-privilege).



Больше MCP-доступа — когда:

конкретная задача требует конкретного инструмента — не «на всякий случай». Каждое подключение — новая граница доверия.

Тот же принцип, что лестница архитектур — применённый к оснастке одного агента: presence paradox показал, что даже файл-инструкция «как ритуал» не работает. Усложняй под конкретный проверяемый триггер.

Итог: механизм → его граница → что делать.

Механизм	Где ломается (граница)	Что делать
Промпт / роль	роль-персона меняет тон, не точность	точность — через контекст и RAG, не через «ты — эксперт»
Chain-of-thought	faithfulness низкая (~25–39%)	проверяй результат, не самообъяснение
RAG	«вернул» ≠ «вернул правильное»	grounding + метрики retrieval + «не знаю» как норма
Fine-tune / PEFT	меняет поведение, не знание; forgetting	PEFT + eval-петля + версионирование; знание → RAG
Агентный цикл	план → act → check → iterate — 4 места отказа	check против внешнего критерия; жёсткий потолок на цикл
Экипировка / память	каждый слот — компромисс, не апгрейд	добавляй слот под требование, с eval
Мульти-агент	р ⁿ : 95%×10 ≈ 60%; +15× токенов	один сильный агент по умолчанию
Безопасность	инъекция × широкие права; ZDR не всё	least-privilege + карта данных по фиче
«Не ИИ вовсе»	детерминированное + верифицируемое	обычный код — дешевле, предсказуемее, аудлируем

Знать инструмент — значит знать его границы. Выбор архитектуры = найти самую нижнюю ступень, которая закрывает требование задачи.

Человек-валидатор + урок MIT NANDA.



Агент делает — человек проверяет результат и факты

против независимого источника истины, НЕ по правдоподобности рассуждения^[1] (самообъяснение модели ≠ контроль — урок про верность рассуждения)

Степень автономности

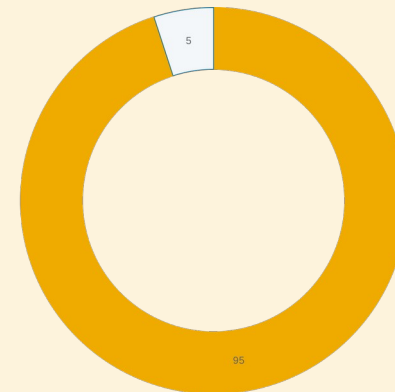
от «человек нажимает кнопку» до «агент уведомляет постфактум»

Область доверия

чтение (легко откатить) vs запись · обратимое vs необратимое

Непрерывный мониторинг

метрики качества постоянно, не разовая проверка на старте



~95%

корпоративных GenAI-пилотов без измеримого возврата инвестиций — корень в разрыве обучения и провале интеграции, не в качестве модели^[2]

«Запустить ИИ» ≠ «получить ценность». Решает архитектурно-интеграционная дисциплина. Иногда правильный ответ — простейшая архитектура или не-ИИ.

Дальше — отрасли. Старт: разработка ПО, ось SDLC × AI.

Лекция 4 берёт тот же аппарат и раскладывает его по этапам жизненного цикла ПО. Каждый этап опирается на якорь из этой лекции:

Требования / дизайн

выбор архитектуры под шаг → лестница из 6 ступеней

Кодирование

coding-агенты через рамку экипировки; провалы как КЛАСС

Тестирование

граница доверия и least-privilege; данные вне ZDR

Эксплуатация

человек-валидатор проверяет результат, а не самообъяснение

Задание — Семинар 3: прогнать чек-лист выбора архитектуры на 3 кейсах (чат / агент / RAG / API).

Что делать: эта рамка — база для Лекций 4–17. На каждой отраслевой лекции прогоняйте тот же чек-лист выбора.



Q&A

Спасибо